

NUSTcord

A Java OOP-Based Communication Platform

Final Semester Project Report

Course: CS-201 Object-Oriented Programming

Institution: National University of Sciences and Technology (NUST)

Semester: 2nd Semester — BS Computer Science

Date: May 2026

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement & Objectives
4. System Architecture
5. Database Design
6. OOP Concepts Applied
7. Features & Modules
8. Challenges Faced
9. Conclusion
10. References

1. Abstract

NUSTcord is a full-stack, Java-based web application designed and developed as a Final Semester Project for the Object-Oriented Programming (OOP) course at the National University of Sciences and Technology (NUST). Inspired by the widely-used communication platform Discord, NUSTcord simulates a structured, community-driven communication environment. The application enables users to register securely, manage friend networks, create and join servers, communicate through text channels, exchange private direct messages, and administrate platform users through a dedicated admin dashboard.

The system is built entirely on core Java technologies — Java 11 Servlets, JavaServer Pages (JSP), and an embedded H2 database — deliberately avoiding heavy frameworks such as Spring Boot or Hibernate. This design choice ensures that all architectural decisions are explicit and traceable to fundamental OOP principles. The application demonstrates encapsulation, abstraction, inheritance, polymorphism, and custom exception handling within a strict four-layer Model-View-Controller (MVC) architecture. Security is addressed through jBCrypt password hashing and session-based authentication enforced by a servlet filter. The result is a fully functional, modular, and maintainable web application that validates the practical application of OOP principles in a real-world software context.

2. Introduction

Modern digital communication platforms are among the most complex categories of software in production today. Systems such as Discord, Slack, and Microsoft Teams handle millions of concurrent users, real-time message streams, permission hierarchies, and community management — all while maintaining security and responsiveness. Behind their polished interfaces lies a disciplined software architecture built upon fundamental principles of object-oriented design.

For the second semester OOP course at NUST, students were challenged to design and implement a substantial Java-based application that demonstrates mastery of OOP concepts in a realistic, integrated context. Rather than producing an isolated academic exercise, the development team chose to build NUSTcord — a Discord-inspired communication platform — as their Final Semester Project. This decision was motivated by the platform's natural alignment with OOP concepts: users, servers, channels, messages, and roles map directly and cleanly to Java objects with well-defined responsibilities, relationships, and behaviors.

NUSTcord is a web application accessible through a browser and deployed on Apache Tomcat. It does not rely on any dependency injection framework, ORM layer, or managed runtime. Every HTTP request is handled by a hand-written servlet. Every database interaction is performed through a raw JDBC PreparedStatement. Every business rule is enforced in an explicitly authored service class. This deliberate constraint transforms the project from a simple application into a rigorous exercise in software architecture.

This report documents the complete design, architecture, OOP application, features, and implementation challenges of the NUSTcord system. It serves as both a technical reference and an academic demonstration of the principles taught throughout the OOP course.

3. Problem Statement & Objectives

3.1 Problem Statement

University-level programming courses frequently teach OOP concepts in isolation — a class demonstrating encapsulation here, an interface demonstrating polymorphism there. While individually instructive, this approach leaves a gap: students rarely experience how these concepts interact and reinforce each other within the architecture of a complete, production-style application.

The core problem this project addresses is: ***How can OOP principles be applied cohesively to design and implement a secure, modular, full-stack web application without relying on frameworks that abstract those principles away?***

Building NUSTcord without Spring Boot, Hibernate, or any managed container forces every architectural decision to be made explicitly, making the role of each OOP concept clearly visible in the codebase.

3.2 Objectives

The project was undertaken with the following specific objectives:

- Apply core OOP concepts — encapsulation, abstraction, inheritance, polymorphism, and exception handling — in a large-scale, integrated Java application rather than isolated demonstrations.
- Implement a strict MVC architecture — separating the data layer (Models + DAOs), business logic layer (Services), and presentation layer (Servlets + JSPs) to achieve high cohesion and low coupling.
- Develop a secure authentication system — using BCrypt password hashing to ensure plain-text passwords are never stored, transmitted in logs, or exposed in any form.
- Design a normalized relational database schema — using H2 with proper foreign key constraints, cascading deletes, and ENUM-typed state fields to maintain data integrity.
- Build a functional, user-facing web interface — using JSP and CSS with a modern dark-mode aesthetic that reflects the Discord design language.
- Demonstrate the SOLID design principles — particularly Single Responsibility and Open/Closed, through the modular class structure of the service and DAO layers.
- Deliver a deployable artifact — packaged as a .war file deployable to any Apache Tomcat 9+ instance, with zero external database configuration required.

4. System Architecture

4.1 Architectural Overview

NUSTcord follows a strict four-layer MVC (Model-View-Controller) architecture. Each layer has a single, well-defined responsibility and communicates only with the layer directly adjacent to it. This design prevents coupling between the UI and the database, ensuring that changes in one layer do not cascade unpredictably through the system.



4.2 Cross-Cutting Components

Beyond the four primary layers, NUSTcord includes several supporting components that operate across the entire request lifecycle:

AuthFilter — A `javax.servlet.Filter` implementation that intercepts every incoming HTTP request before it reaches a servlet. It inspects the session for a valid `userId` attribute. If the attribute is absent, the filter immediately redirects the request to `login.jsp`. Only the `/login`, `/register`, and static resource paths are whitelisted.

AppInitListener — A `ServletContextListener` that fires once when Tomcat deploys the application. It executes the full database schema creation SQL and checks whether the two seed accounts (dummy and admin) already exist, inserting them with pre-computed BCrypt hashes if they do not.

DBConnection — A utility class providing a singleton JDBC connection to the embedded H2 database file. All DAO classes retrieve their connection through this utility.

PasswordUtil — A thin wrapper around the `org.mindrot.jbcrypt.BCrypt` library exposing two static methods: `hashPassword(String plain)` and `checkPassword(String plain, String hash)`. This abstraction ensures BCrypt is always used consistently.

AuthException — A custom checked exception in the `com.nustcord.exception` package. Thrown by `AuthService` whenever registration or login fails due to a business rule violation (e.g., duplicate username, password too short, wrong credentials).

4.3 Request-Response Flow

The following sequence illustrates how a typical login request travels through the system:

```
1. Browser sends POST /login with form fields: username, password
2. AuthFilter checks session → no userId → request is for /login, so ALLOW
3. LoginServlet.doPost() is invoked
   └─ Reads username and password from request.getParameter()
4. LoginServlet calls authService.login(username, password)
5. AuthService.login():
   └─ calls userDao.getUserByUsername(username)
       └─ UserDao executes: SELECT * FROM users WHERE username = ?
           └─ Returns a User object (or null)
       └─ Calls PasswordUtil.checkPassword(plain, user.getPasswordHash())
       └─ Returns authenticated User object on success
       └─ Throws AuthException on failure
6. LoginServlet receives User object
   └─ Sets session attributes: userId, username
   └─ Calls statusDAO.setStatus(userId, "Online")
   └─ Redirects to /dashboard (or /admin if username == "admin")
7. Browser follows redirect to /dashboard
8. DashboardServlet.doGet() is invoked
   └─ Reads userId from session
   └─ Fetches data via service/DAO calls
   └─ Sets attributes on request object
   └─ Forwards to dashboard.jsp
9. dashboard.jsp renders HTML using request attributes
   └─ Browser displays the dashboard page
```

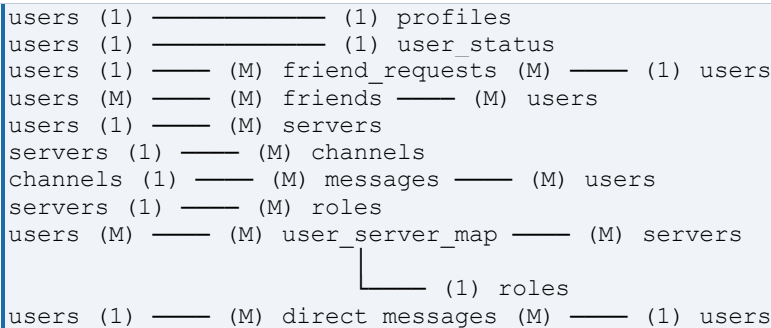
5. Database Design

5.1 Overview

The NUSTcord database is an embedded H2 relational database stored as a local file (nustcord_db.mv.db). H2 is configured in MySQL-compatibility mode, which ensures that SQL syntax, ENUM types, and constraint behavior closely match what would be expected in a production MySQL deployment. The schema is defined in schema.sql and executed automatically by ApplnitListener on first startup.

The schema consists of eleven tables organized into four logical groups: User Identity, Social Graph, Community (Servers/Channels), and Messaging.

5.2 Entity-Relationship Overview



5.3 Table Definitions

users

The central identity table. Every other table references it through foreign keys.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
username	VARCHAR(50)	UNIQUE NOT NULL
email	VARCHAR(100)	UNIQUE NOT NULL
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

profiles

Optional user profile data, separated from users to keep the core identity table lean.

Column	Type	Constraints
user_id	INT	PRIMARY KEY, FK → users(id) ON DELETE CASCADE
display_name	VARCHAR(50)	—
bio	TEXT	—

user_status

Tracks each user's current activity state.

Column	Type	Constraints
user_id	INT	PRIMARY KEY, FK → users(id) ON DELETE CASCADE
status	ENUM	('Online', 'Offline', 'Busy', 'Away') DEFAULT 'Offline'
last_updated	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

friend_requests

Manages the lifecycle of friendship invitations.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
sender_id	INT	FK → users(id) ON DELETE CASCADE
receiver_id	INT	FK → users(id) ON DELETE CASCADE
status	ENUM	('PENDING', 'ACCEPTED', 'REJECTED', 'CANCELLED') DEFAULT 'PENDING'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
—	UNIQUE KEY	(sender_id, receiver_id)

friends

Records established bidirectional friendships. A pair (A,B) is stored once; queries check both orderings.

Column	Type	Constraints
user_id1	INT	FK → users(id) ON DELETE CASCADE
user_id2	INT	FK → users(id) ON DELETE CASCADE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
—	PRIMARY KEY	(user_id1, user_id2)

servers

Community spaces created and owned by users.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
name	VARCHAR(100)	NOT NULL
owner_id	INT	FK → users(id) ON DELETE CASCADE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

channels

Text or voice channels belonging to a server.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
server_id	INT	FK → servers(id) ON DELETE CASCADE
name	VARCHAR(100)	NOT NULL
type	ENUM	('TEXT','VOICE') NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

messages

Individual messages posted in a text channel.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
channel_id	INT	FK → channels(id) ON DELETE CASCADE
sender_id	INT	FK → users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
created at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

roles

Named roles with permission sets, scoped to a specific server.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
server_id	INT	FK → servers(id) ON DELETE CASCADE
name	VARCHAR(50)	NOT NULL
permissions	TEXT	—

user_server_map

Many-to-many join table linking users to servers with optional role assignment.

Column	Type	Constraints
user_id	INT	FK → users(id) ON DELETE CASCADE
server_id	INT	FK → servers(id) ON DELETE CASCADE
role_id	INT	FK → roles(id) ON DELETE SET NULL
—	PRIMARY KEY	(user_id, server_id)

direct_messages

Private messages exchanged between two users.

Column	Type	Constraints
id	INT	AUTO_INCREMENT PRIMARY KEY
sender_id	INT	FK → users(id) ON DELETE CASCADE
receiver_id	INT	FK → users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

5.4 Design Decisions

Cascading Deletes — Every foreign key references its parent with ON DELETE CASCADE. Deleting a user automatically removes their profile, status, friend relationships, messages, server memberships, and owned servers. Deleting a server cascades to its channels, roles, memberships, and all messages in those channels.

Separation of users and profiles — The core users table contains only authentication-critical data. Display preferences are stored in a separate profiles table in a one-to-one relationship, adhering to the Single Responsibility Principle at the database level.

ENUM State Fields — The status and type columns use SQL ENUM types rather than plain VARCHAR, enforcing valid value constraints at the database level.

Composite Primary Keys for Join Tables — The friends and user_server_map tables use composite primary keys rather than surrogate auto-increment keys, naturally preventing duplicate memberships and friendships.

6. OOP Concepts Applied

6.1 Encapsulation

Encapsulation is the OOP principle of bundling data (fields) and the methods that operate on that data within a single unit (class), while restricting direct external access to the internal state. In NUSTcord, encapsulation is applied consistently across all eleven model classes.

Consider `User.java`. The class declares all its fields as private: `id`, `username`, `email`, `passwordHash`, and `createdAt`. None of these fields can be accessed or modified by any external class directly. Instead, the class exposes controlled public getter and setter methods for each field.

```
// ENCAPSULATION in User.java
public class User {
    private int id;
    private String username;
    private String email;
    private String passwordHash; // Never exposed for display
    private Timestamp createdAt;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPasswordHash() { return passwordHash; }
    // No setPasswordHash exposed publicly – modification goes through AuthService
}
```

This is particularly important for the `passwordHash` field. JSP views and servlets have access to `User` objects, but because `passwordHash` is private and the getter is documented as internal-use only, it is never accidentally rendered in a page or logged. The encapsulated design acts as a guardrail against security mistakes.

6.2 Abstraction

Abstraction is the principle of hiding complex implementation details behind a simplified interface. In NUSTcord, abstraction is realized through the service layer, which acts as a clean boundary between the controllers (servlets) and the data access layer (DAOs).

`LoginServlet` does not know how authentication works internally. It only knows that it can call `authService.login(username, password)` and receive either a `User` object or an `AuthException`.

```
// ABSTRACTION in LoginServlet.java
AuthService authService = new AuthService();
try {
    User user = authService.login(username, password); // All complexity hidden
}
```

```
        session.setAttribute("userId", user.getId());
        response.sendRedirect("dashboard");
    } catch (AuthException e) {
        request.setAttribute("error", e.getMessage());
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }
```

This layered abstraction means that if the database is ever migrated from H2 to PostgreSQL, only the DAO implementations need to change. Servlets and services remain untouched.

6.3 Inheritance

Inheritance allows a class to acquire the fields and methods of a parent class, enabling code reuse and establishing an 'is-a' relationship. NUSTcord uses inheritance in two distinct contexts.

Servlet Inheritance — Every servlet extends `javax.servlet.http.HttpServlet`, inheriting the `service()` method dispatcher and gaining the ability to override `doGet()` and `doPost()`.

Exception Inheritance — `AuthException` extends `java.lang.Exception`, creating a domain-specific exception type compatible with Java's standard exception handling infrastructure. This makes error handling an explicit architectural contract enforced by the compiler.

```
// INHERITANCE - ServerServlet extends HttpServlet
public class ServerServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handle GET requests for server listing
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handle POST requests for server creation
    }
}

// INHERITANCE - AuthException extends Exception
public class AuthException extends Exception {
    public AuthException(String message) {
        super(message); // Delegates to parent Exception constructor
    }
}
```

6.4 Polymorphism

Polymorphism allows objects of different types to be treated through a common interface. All thirteen servlets inherit `doGet()` and `doPost()` from `HttpServlet`, with each servlet overriding only the methods relevant to its function:

```
// POLYMORPHISM — same method name, different behaviour per class

// In LoginServlet:
protected void doPost(HttpServletRequest req, HttpServletResponse res) {
    // Authenticates user credentials, creates session
}
// In RegisterServlet:
protected void doPost(HttpServletRequest req, HttpServletResponse res) {
    // Validates and creates a new user account
}
// In MessageServlet:
protected void doPost(HttpServletRequest req, HttpServletResponse res) {
    // Persists a new message to a channel
}
```

When Tomcat receives an HTTP POST request, it calls `doPost()` on whichever servlet is mapped to the URL. The runtime determines which implementation to invoke based on the actual type of the servlet object — this is runtime polymorphism.

6.5 Exception Handling

NUSTcord implements a custom exception handling strategy centered on `AuthException`. Rather than propagating raw `SQLException` objects to the view layer, the service layer catches low-level exceptions and re-throws them as `AuthException` with controlled, user-appropriate messages.

```
// EXCEPTION HANDLING in AuthService.java
public boolean register(String username, String email, String password)
    throws AuthException {
    if (username == null || username.trim().isEmpty() || password.length() < 6) {
        throw new AuthException("Invalid username or password must be at least 6
characters.");
    }
    if (userDAO.getUserByUsername(username) != null) {
        throw new AuthException("Username already exists.");
    }
    try {
        String hash = PasswordUtil.hashPassword(password);
        return userDAO.registerUser(new User(0, username, email, hash, null));
    } catch (SQLException e) {
        if (e.getMessage().toLowerCase().contains("unique")) {
            throw new AuthException("Username or Email already taken.");
        }
        throw new AuthException("A database error occurred: " + e.getMessage());
    }
}
```

The exception hierarchy is clean: AuthException signals a known, expected failure (bad input, duplicate user, wrong password). Raw IOException and ServletException propagate through the servlet framework for unexpected, unrecoverable errors.

6.6 SOLID Principles

Single Responsibility (S) — Each class has exactly one reason to change. User.java changes only if the user data model changes; UserDAO.java changes only if the SQL queries change; AuthService.java changes only if authentication business rules change.

Open/Closed Principle (O) — The system is open for extension and closed for modification. Adding a new feature requires adding new classes without modifying existing ones.

Dependency Inversion (D) — Servlets depend on service class abstractions, not on DAO implementations. A FriendServlet calls friendService.sendRequest() and does not care whether FriendService uses FriendRequestDAO, FriendsDAO, or both.

7. Features & Modules

7.1 Secure Authentication & User Identity

Authentication is handled by AuthService, LoginServlet, RegisterServlet, and LogoutServlet. When a user registers, their plain-text password is passed to PasswordUtil.hashPassword(), which calls BCrypt.hashpw() with a generated salt. The resulting 60-character hash is what is stored in the users table — the plain-text password is discarded immediately after hashing.

During login, PasswordUtil.checkPassword() calls BCrypt.checkpw() to compare the submitted plain-text password against the stored hash without ever reconstructing the original password. Sessions are managed via HttpSession. On logout, LogoutServlet calls session.invalidate() and sets the user's status to Offline.

7.2 Admin Dashboard

The admin dashboard (adminDashboard.jsp, served by AdminServlet) is exclusively accessible to the admin account. AdminDAO executes a complex SQL query JOINing the users, profiles, user_status, friends, and user_server_map tables to produce a comprehensive user record displaying each user's ID, username, email, display name, registration date, current status, friend count, server count, and last activity timestamp.

Client-side JavaScript implements live search across username, email, and display name fields, a dropdown filter for status, and sortable column headers — all processed in the browser without additional server round-trips.

7.3 Friend Networking

The friend system operates as a state machine managed by FriendRequestDAO and FriendsDAO, orchestrated by FriendService and routed through FriendServlet. A user can send a friend request by exact username, creating a PENDING row in friend_requests. On acceptance, FriendService atomically updates the row to ACCEPTED and inserts a new row into the friends table.

The friends table uses a composite primary key (user_id1, user_id2), always storing the lower ID first, preventing duplicate bidirectional entries.

7.4 Server Management & Roles

Any authenticated user can create a server through `ServerServlet`. On creation, `ServerService` performs three sequential DAO operations: it inserts the server record, creates a default Admin role, creates a default #general TEXT channel, and adds the creating user to `user_server_map` with the Admin role.

The server list page (`serverList.jsp`) shows all existing servers and whether the current user is a member. Non-members can join through `JoinServerServlet`. Server owners can manage members, assign roles, and manage channels through `serverSettings.jsp`.

7.5 Channels & Messaging

Each server contains one or more text channels managed by `ChannelServlet`. When a user selects a channel, `channelView.jsp` is rendered with messages retrieved by `MessageDAO`, ordered chronologically. `MessageServlet` handles both fetching (GET) and posting (POST) messages. The `messages.jsp` view uses JavaScript auto-scroll to position the viewport at the latest message.

7.6 Direct Messaging

Direct messages are private, persistent, 1-on-1 conversations between friends. `DirectMessageDAO` executes a query selecting all rows where `(sender_id = A AND receiver_id = B) OR (sender_id = B AND receiver_id = A)`, ordered by `created_at` — retrieving the complete bidirectional conversation in a single query.

7.7 User Profiles & Status

Each user has an optional profile managed through `ProfileServlet` storing a `display_name` (shown in messages instead of the raw username) and a bio. Status is updated to Online on login, Offline on logout, and can be manually changed through a dropdown in `settings.jsp`.

7.8 Application Lifecycle (`AppInitListener`)

`AppInitListener` implements `ServletContextListener` and performs the following idempotent initialization sequence on each startup:

- Executes all `CREATE TABLE IF NOT EXISTS` statements from the embedded schema definition.
- Checks for the existence of the dummy user via `UserDAO.getUserByUsername("dummy")`.
- If absent, inserts the dummy user with a pre-computed BCrypt hash of "dummy".
- Checks for the existence of the admin user.
- If absent, inserts the admin user with a pre-computed BCrypt hash of "password1".

The use of IF NOT EXISTS in the DDL and existence checks before seed inserts makes this process safe to run on every startup, including restarts after the database has already been initialized.

8. Challenges Faced

8.1 Session Management Without a Framework

In a Spring MVC application, session management is largely handled by the framework. In NUSTcord, every servlet manually reads from and writes to the `HttpSession`. Ensuring consistent attribute naming (`userId`, `username`) across thirteen servlets required establishing naming conventions early and adhering to them strictly. A single typo — `getUserId` vs `getUserid` — would cause silent null-pointer behaviour that is difficult to trace.

8.2 Relational Schema Design with Cascading Constraints

Designing the database schema with correct cascading delete behaviour required careful analysis of the entity relationship graph. Deleting a user must cascade to their profile, status, friend requests, friendship records, owned servers (which cascade further to channels, roles, messages, and memberships), server memberships, messages, and direct messages. H2's strict foreign key enforcement meant that any missing cascade rule immediately surfaced as a constraint violation during testing.

8.3 Secure BCrypt Integration

Integrating BCrypt correctly required understanding its API in detail. Key challenges included ensuring `hashpw()` was called before the `User` object was constructed, recognising that `checkpw()` returns a boolean and never throws for wrong passwords, and ensuring log statements never inadvertently included the plain-text password. `PasswordUtil` was created to encapsulate these concerns in one place.

8.4 Maintaining Strict MVC Layer Separation

Without a dependency injection container, it was tempting to call DAO methods directly from servlets when a quick fix was needed. Maintaining discipline — always routing through the service layer — required consistent code review and refactoring. Several early implementations had servlets calling DAO methods directly, which were later corrected to pass through the appropriate service class.

8.5 Bidirectional Friendship Logic

The friends table uses a composite primary key (`user_id1`, `user_id2`). To prevent both (A, B) and (B, A) from existing simultaneously, the application enforces a convention that the lower user ID is always stored as `user_id1`. This means all friendship queries must check both orderings. Forgetting this bidirectionality in any one query would cause some friendships to appear missing.

8.6 Idempotent Application Initialization

Ensuring that `AppInitListener` could run on every Tomcat startup without creating duplicate tables or duplicate seed users required using `CREATE TABLE IF NOT EXISTS` in the DDL and explicit existence checks before seed inserts. An early version used raw `CREATE TABLE` statements, causing the application to fail on every restart after the first deployment.

8.7 JSP and CSS Without a Component Framework

Building a cohesive, dark-mode UI across fourteen JSP pages using only vanilla CSS required a disciplined approach to shared styles. Reusable JSP fragments (header navigation, sidebar, user controls) were extracted into an `includes/` directory and imported via `<jsp:include>` tags. A global stylesheet enforced the colour palette (`#1a1b2e` background, `#e0e0ff` text, `#5865f2` accent) across all pages.

9. Conclusion

The development of NUSTcord as a Final Semester Project delivered outcomes that exceeded the narrow requirements of producing a working application. By choosing to build a communication platform without the aid of dependency injection frameworks or ORM tools, the project forced every architectural decision to be made deliberately and explicitly — transforming each decision into a learning opportunity.

The strict four-layer MVC architecture proved its value repeatedly throughout development. When the database schema required modification, only DAO classes needed updating. When authentication rules changed, only AuthService was affected. When the UI required redesign, JSP files could be changed without touching any Java class. This independence of layers — achieved through disciplined abstraction and encapsulation — is the practical payoff of OOP principles.

BCrypt integration demonstrated that security is not an add-on but an architectural concern. By centralising password hashing in PasswordUtil and making AuthService the sole arbiter of authentication, the system ensures that no code path can accidentally expose or misuse credentials.

The custom exception hierarchy (AuthException extends Exception) illustrated how inheritance can be used not just for code reuse but for communication — making the failure modes of a system explicit and contractual through the Java compiler's checked exception enforcement.

Ultimately, NUSTcord demonstrates that OOP is not a set of academic constraints but a toolkit for managing complexity. The skills exercised in this project — layered architecture design, relational database modelling, secure authentication, session management, and CSS-driven UI construction — are directly applicable to professional Java web development, regardless of which framework is ultimately used in a production environment.

10. References

- [1] Oracle Corporation. Java Servlet Technology - Java EE 8 Documentation. Oracle. [Online]. Available: <https://javaee.github.io/servlet-spec/>. [Accessed: May 2026].
- [2] H2 Database Engine. H2 Database Engine — Documentation. H2 Group. [Online]. Available: <https://www.h2database.com/html/main.html>. [Accessed: May 2026].
- [3] Mindrot.org. jBCrypt — A Java implementation of OpenBSD's Blowfish password hashing code. [Online]. Available: <https://www.mindrot.org/projects/jBCrypt/>. [Accessed: May 2026].
- [4] Apache Software Foundation. Apache Tomcat 9 Documentation. Apache. [Online]. Available: <https://tomcat.apache.org/tomcat-9.0-doc/>. [Accessed: May 2026].
- [5] C. S. Horstmann. Core Java Volume I — Fundamentals, 11th ed. Prentice Hall, 2018.
- [6] R. C. Martin. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- [7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [8] Apache Software Foundation. Apache Maven Project Documentation. [Online]. Available: <https://maven.apache.org/guides/>. [Accessed: May 2026].